Quefy: product description

Quefy is a jukebox for parties. One person hosts a room, friends join from their phones (QR code or a 6-letter code, no install needed on the web), search for songs and add them to a shared queue. The queue is ordered fairly, so nobody can fill it with their own music, and the crowd can vote to skip a bad song. An AI part checks that songs fit the mood the host chose and adds songs when the queue is empty.

This document covers task 3: functional specification, UML and identity management.

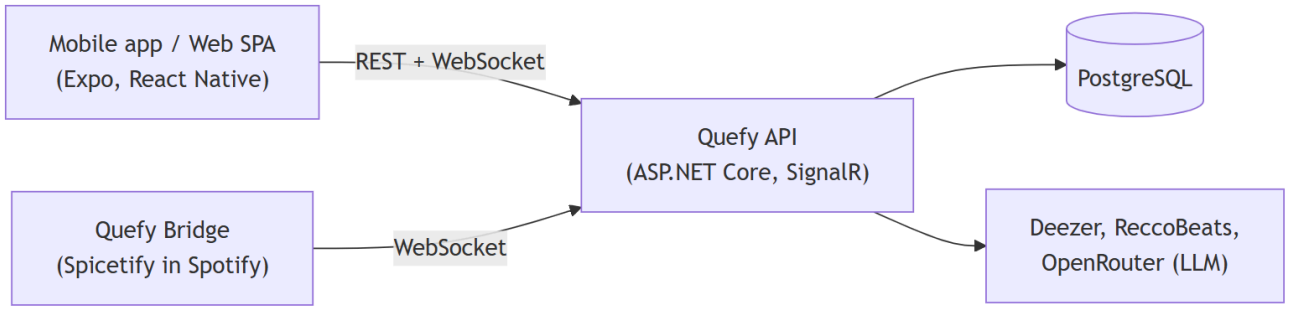
1. Goals

- Start a party in under 30 seconds, without sign-up.
- Guests join without installing anything.
- Everybody gets a fair share of airtime.
- The crowd, not one person, decides what is played.
- The music stays on the chosen vibe without the host babysitting it.

Out of scope: payments, accounts with e-mail/password, App Store release, public discovery of parties.

2. High-level architecture

Quefy is a modular monolith: one ASP.NET Core API with PostgreSQL, a React Native (Expo) client that also runs as a web page, and a small bridge that controls Spotify on the host's computer. More detail is in document 04.



3. Actors and personas

Actor	Description
Host	Creates the party, owns it, can end it and change roles
Co-host	A guest the host promoted; can moderate the queue and rules
Guest	Joins with a code, adds songs, votes, votes to skip
Speaker	Spotify bridge or a phone/browser that actually plays the music
Quefy DJ	System user (AI) that adds songs when the queue is empty
Operator	Runs the server and watches health and metrics

Personas:

- **Olya, 21, host.** Throws a flat party and doesn't want to be the DJ all night. Wants to start fast and keep the music fitting.
- **Max, 23, guest.** Always has ten songs he wants to play. Quefy makes him wait his turn, which he accepts because the others do too.
- **Ira, 35, guest on an Android phone.** Doesn't want to install anything. Opens the link, types a name, adds a song.

4. Functional requirements

ID	Requirement
FR-01	The system creates an anonymous account for a device from a display name; no e-mail or password.
FR-02	A user can see and change their name, colour and language.
FR-03	Sessions are kept alive with rotating refresh tokens; reusing an old token revokes the session.
FR-04	A user can host a party with a name, a vibe and rules. The party gets a 6-character code and a QR code.
FR-05	A user can preview and join a party by code, QR code or link, also from a browser.
FR-06	The host can change the rules (queue limits, cooldown, skip ratio, explicit tracks, vibe policy). Two people editing at once must not overwrite each other.
FR-07	The host can promote a guest to co-host, kick or ban a member, and end the party.
FR-08	A member can search the music catalog (Deezer).
FR-09	A member can add a track. The rules are checked: duplicate, recently played, too long, explicit, per-guest limit, cooldown.
FR-10	The queue is ordered by the Fair-Play score (votes, waiting time, number of own songs, recent airtime) and each item shows why it is where it is.
FR-11	A member can up-vote or down-vote a waiting track, but not their own.
FR-12	Hosts and co-hosts can remove a track, pin it as "play next", and approve or reject held tracks.
FR-13	Members can vote to skip the current track (Fair Skip). With a ratio of 0.5 a strict majority of listeners is needed. The song's own requester can skip immediately.
FR-14	The host can pause, resume and skip. A history of played songs is available.
FR-15	A speaker is connected by a pairing code (Spotify on a computer) or by using the phone/browser itself.
FR-16	The server drives playback: it starts the next track by itself, without a gap on Spotify.
FR-17	The host sets a vibe from a preset or from free text (compiled by an LLM).
FR-18	Every added track gets a vibe verdict (fits / borderline / off). Depending on the policy, off tracks are only marked, or held for approval.
FR-19	When the queue runs out, the AI DJ adds up to 2 songs that match the crowd's taste and the vibe. A member can also ask the DJ for suggestions.
FR-20	All members see changes live (queue, now playing, members) and get personal notices.
FR-21	The UI is available in English and Ukrainian.

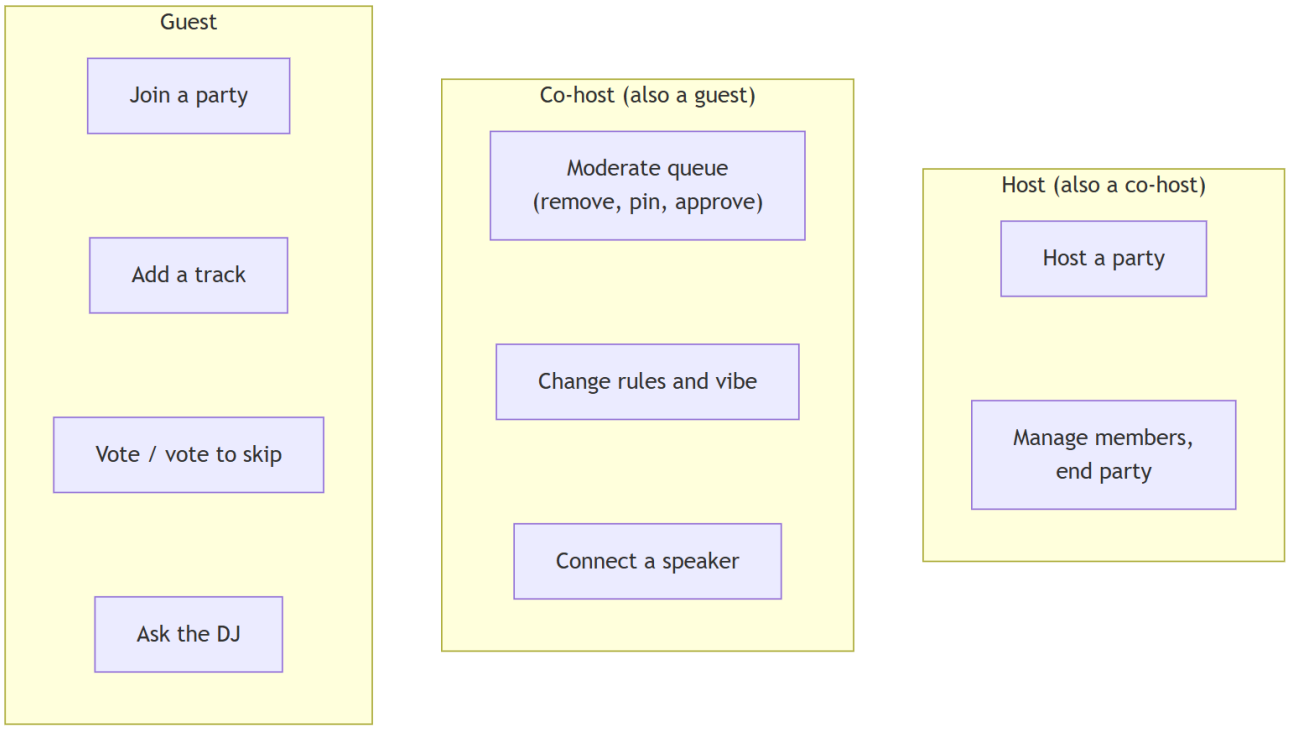
ID	Requirement
FR-22	Old data is deleted automatically; idle parties end after 12 hours.

5. Non-functional requirements

ID	Requirement	Target
NFR-01	Performance	REST p95 under 300 ms (without upstream calls); a change reaches clients in under 500 ms
NFR-02	Security	Every call except public ones needs a JWT; tokens and pairing codes are stored hashed; rate limits on public endpoints
NFR-03	Correctness	No double adds, no double skips when requests arrive at the same time
NFR-04	Resilience	If Deezer, ReccoBeats or the LLM fail, the party continues with a fallback
NFR-05	Availability	Demo: best effort on one PC. Production target 99.5 %
NFR-06	Usability	A new host has a room with a QR code in 30 seconds; guests need no install
NFR-07	Accessibility	Screen reader labels, large touch targets, readable contrast
NFR-08	Observability	Metrics, traces, structured logs and health endpoints
NFR-09	Maintainability	Domain rules are plain code with unit tests; external services sit behind interfaces

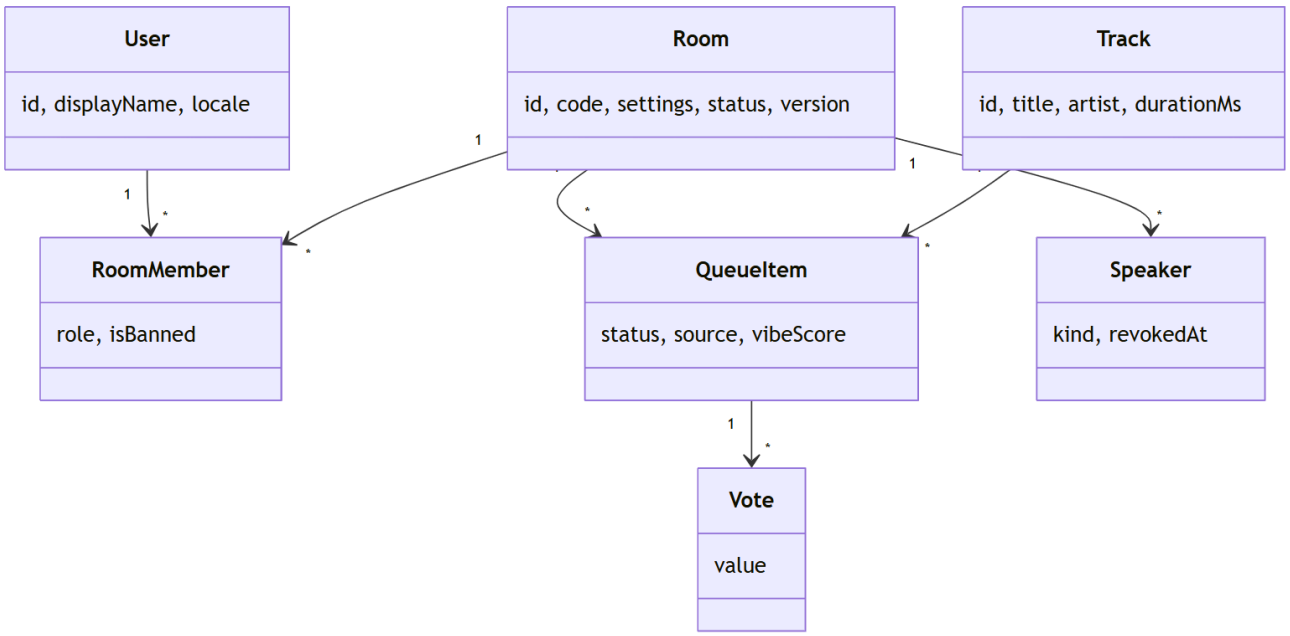
6. UML

Use cases

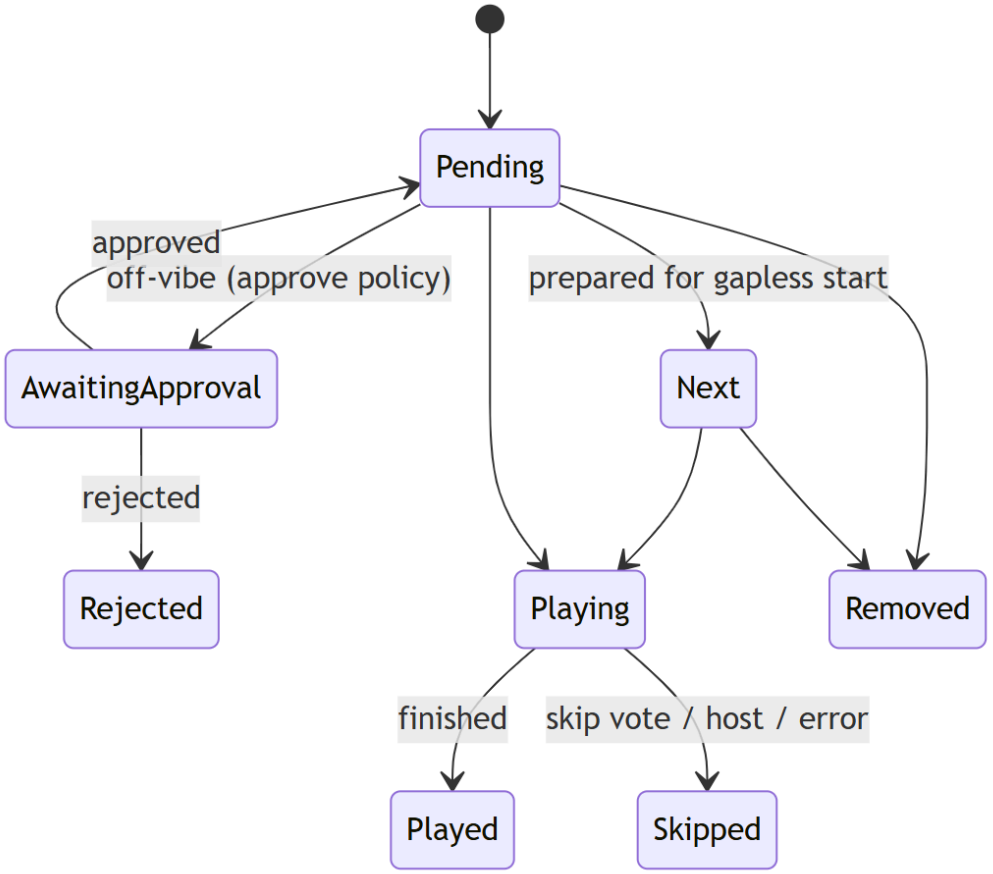


The host and co-host can also do everything a guest can.

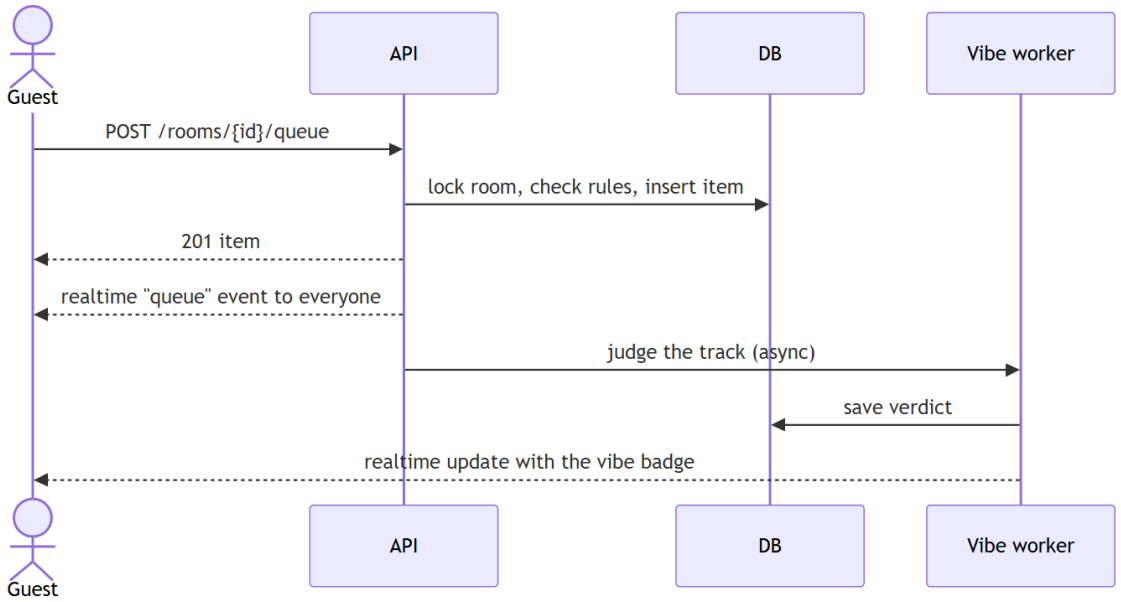
Main classes



Queue item states



Adding a track



7. Identity management

There are no passwords. On first launch the app registers the device silently and gets a guest name like "Guest 4821"; the user can pick a real name when they host or join.

Item	How it works
User id	UUIDv7, created at registration
Access token	JWT (HS256), 30 minutes, claims `sub`, `typ=user`, `sid`
Refresh token	Random value, 90 days, stored only as a SHA-256 hash. Works once; each refresh returns a new pair
Theft detection	A refresh token that was already used means it leaked, so the whole token family is revoked
Speaker token	JWT with `typ=speaker` and a `room_id`, 24 hours, valid for the speaker hub only
Pairing code	6 characters, 5 minutes, single use, stored as a hash
Storage on device	iOS Keychain; browser local storage
Cleanup	Anonymous accounts with no party that were not seen for 180 days are deleted

Roles inside a room are checked on the server for every request:

Action	Guest	Co-host	Host
Search, add, vote, vote to skip	yes	yes	yes
Remove own track	yes	yes	yes
Remove any track, pin, approve/reject	no	yes	yes
Pause, skip, change rules and vibe, connect speaker	no	yes	yes
Kick or ban a guest	no	yes	yes
Promote/demote, kick a co-host, end party	no	no	yes